

```
1. quicksort <- function(arr) {
```

```
  if (length(arr) <= 1) {
```

```
    return(arr)
```

```
  }
```

```
  pivot <- arr[1]
```

```
  left <- arr[arr < pivot]
```

```
  middle <- arr[arr == pivot]
```

```
  right <- arr[arr > pivot]
```

```
  return(c(quicksort(left), middle, quicksort(right)))
```

```
}
```

```
arr <- c(45, 12, 89, 33, 7, 56, 21)
```

```
cat("Original Array:\n")
```

```
print(arr)
```

```
sorted_arr <- quicksort(arr)
```

```
cat("Sorted Array:\n")
```

```
print(sorted_arr)
```

```
output
```

```
Original Array:
```

```
> print(arr)
```

```
[1] 45 12 89 33 7 56 21
```

```
>
```

```
2. # Create a new node
```

```
createNode <- function(data) {
```

```
list(data = data, left = NULL, right = NULL)
}
```

Insert a node into BST

```
insertNode <- function(root, data) {
```

```
  if (is.null(root)) {
    return(createNode(data))
  }
```

```
  if (data < root$data) {
    root$left <- insertNode(root$left, data)
  } else {
    root$right <- insertNode(root$right, data)
  }
```

```
  return(root)
}
```

Inorder Traversal (Left, Root, Right)

```
inorder <- function(root) {
```

```
  if (!is.null(root)) {
    inorder(root$left)
    cat(root$data, " ")
    inorder(root$right)
  }
```

```
}
```

Preorder Traversal (Root, Left, Right)

```
preorder <- function(root) {
```

```
  if (!is.null(root)) {
```

```
    cat(root$data, " ")
    preorder(root$left)
    preorder(root$right)
  }
}
```

Postorder Traversal (Left, Right, Root)

```
postorder <- function(root) {
  if (!is.null(root)) {
    postorder(root$left)
    postorder(root$right)
    cat(root$data, " ")
  }
}
```

Insert values into BST

```
values <- c(50, 30, 70, 20, 40, 60, 80)
```

```
root <- NULL
```

```
for (i in values) {
  root <- insertNode(root, i)
}
```

Display Traversals

```
cat("Inorder Traversal:\n")
```

```
inorder(root)
```

```
cat("\n\nPreorder Traversal:\n")
```

```
preorder(root)
```

```
cat("\n\nPostorder Traversal:\n")
```

```
postorder(root)
```

Inorder Traversal:

```
> inorder(root)
```

```
20 30 40 50 60 70 80 >
```

```
> cat("\n\nPreorder Traversal:\n")
```

Preorder Traversal:

```
> preorder(root)
```

```
50 30 20 40 70 60 80 >
```

```
> cat("\n\nPostorder Traversal:\n")
```

Postorder Traversal:

```
> postorder(root)
```

```
20 40 30 60 80 70 50
```

```
> cat("\n\nPostorder Traversal:\n")
```

3.

Function Used For	Input	Output
apply() Apply a function to rows or columns of a matrix/array	Matrix/Array	Vector, Matrix, or Array
lapply() Apply a function to each element of a list	List	List
sapply() Simplified version of lapply()	List/Vector	Vector or Matrix
tapply() Apply a function to grouped data	Vector + Factor	Group-wise result

1. apply()

Definition:

apply() applies a function to the rows or columns of a matrix.

Syntax:

```
apply(X, MARGIN, FUN)
```

- X → Matrix
- MARGIN = 1 → Rows
- MARGIN = 2 → Columns
- FUN → Function

Example:

```
mat <- matrix(c(1,2,3,4,5,6), nrow=2)
```

```
print(mat)
```

```
apply(mat, 1, sum)
```

```
apply(mat, 2, sum)
```

Output:

```
      [,1] [,2] [,3]  
[1,]   1   3   5  
[2,]   2   4   6
```

Row Sum:

```
9 12
```

Column Sum:

```
3 7 11
```

2. lapply()**Definition:**

`lapply()` applies a function to each element of a list and returns a **list**.

Syntax:

```
lapply(list, function)
```

Example:

```
x <- list(a=1:3, b=4:6)
```

```
lapply(x, sum)
```

Output:

```
$a  
[1] 6
```

```
$b
```

```
[1] 15
```

3. sapply()

Definition:

sapply() is similar to lapply(), but returns a simplified output (vector or matrix).

Example:

```
x <- list(a=1:3, b=4:6)
```

```
sapply(x, sum)
```

Output:

```
a b
```

```
6 15
```

4. tapply()

Definition:

tapply() applies a function to subsets of a vector grouped by a factor.

Syntax:

```
tapply(vector, factor, function)
```

Example:

```
marks <- c(80,90,70,60,85,75)
```

```
group <- c("A","A","B","B","A","B")
```

```
tapply(marks, group, mean)
```

Output:

```
A B
```

```
85 68.33
```

Comparison Table

Feature	apply()	lapply()	sapply()	tapply()
Works on	Matrix/Array	List	List/Vector	Grouped Vector

Feature	apply()	lapply()	sapply()	tapply()
Returns	Vector/Matrix	List	Vector/Matrix	Group-wise values
Main Purpose	Row/Column operations	Process list elements	Simplified list output	Group-wise calculations
Example Function	sum()	sum()	sum()	mean()

Conclusion

- **apply()** → Used for **rows and columns of matrices**.
- **lapply()** → Used for **lists** and returns a **list**.
- **sapply()** → Similar to lapply(), but returns a **simplified vector or matrix**.
- **tapply()** → Used for **group-wise operations** such as finding the mean or sum of each group.